# PRACTICAL- 11

Student Management CRUD with Search, Filter, and MySQL Statements

**AIM / Problem Definition:** Create a student management module to add, view, edit, delete, search and filter student records. All database operations must use MySQLi / PDO statements and clear success/failure messages.

**Tools / Technology Used:** PHP 8.x, MySQL / phpMyAdmin, Apache / XAMPP, HTML5, CSS3, JavaScript, AJAX / Fetch API

## 1. Project Implementation & CRUD Subsystem Architecture

### 1.1 Project Overview (StudentHub — Student Registry Management)

Following the implementation of role-based session authentication in Practical 10, Practical 11 develops the core Administrative Student Registry Management module for the StudentHub University Portal. This data-driven administrative subsystem empowers authenticated institutional administrators to maintain complete lifecycle control over student records. The module encapsulates full CRUD capabilities (Create new student profile, Read/SELECT student directories with pagination, Update existing academic attributes, and Delete obsolete records with confirmation guards). Additionally, the registry provides real-time multi-column search (by student ID, name, email, roll number) and dynamic departmental filtering (Computer Engineering, IT, AI & Data Science, Civil, Electrical), backed by robust server-side MySQL statements and clear user feedback notifications.

### 1.2 Technical Architecture & Database Operation Workflow

The student management subsystem implements secure data-access patterns, defensive error handling, and responsive frontend-backend interaction:

- **C — Create (INSERT Record):** Administrators can register new students directly from the administrative portal via a dedicated modal dialog ('Add New Student Record'). The backend validates inputs server-side, generates formatted unique student codes (STU-035), checks duplicate emails/roll numbers, hashes temporary passwords using bcrypt, and executes parameterized INSERT statements into the 'students' table.
- **R — Read (SELECT & Pagination):** The registry renders student profiles in a responsive tabular layout. To optimize database throughput and browser memory, the backend queries the database using LIMIT and OFFSET clauses (8 records per page across 27+ seeded students). Individual student profiles can also be inspected via a focused 'Student Registry Details' inspection modal.
- **U — Update (UPDATE Operation):** Administrators can modify student credentials—including full name, contact phone, roll number, department, semester, and academic notes—via the 'Edit Student' interface. Changes are sanitized, validated, and updated in MySQL using parameterized UPDATE statements, automatically refreshing the updated_at timestamp.
- **D — Delete (DELETE Operation with Confirmation):** To prevent catastrophic accidental data loss, delete operations strictly require explicit two-step user confirmation through a modal warning dialog ('Confirm Student Deletion'). Upon confirmation, a targeted DELETE query removes the record from the database, cascading clean removal across dependent associations.

• **Real-Time Search & Departmental Filter:** The registry implements dynamic search utilizing SQL LIKE clauses (e.g., WHERE first_name LIKE ? OR last_name LIKE ? OR email LIKE ? OR roll_number LIKE ?) combined with selective dropdown filtering by academic department, returning matching subsets instantly without disrupting pagination state.

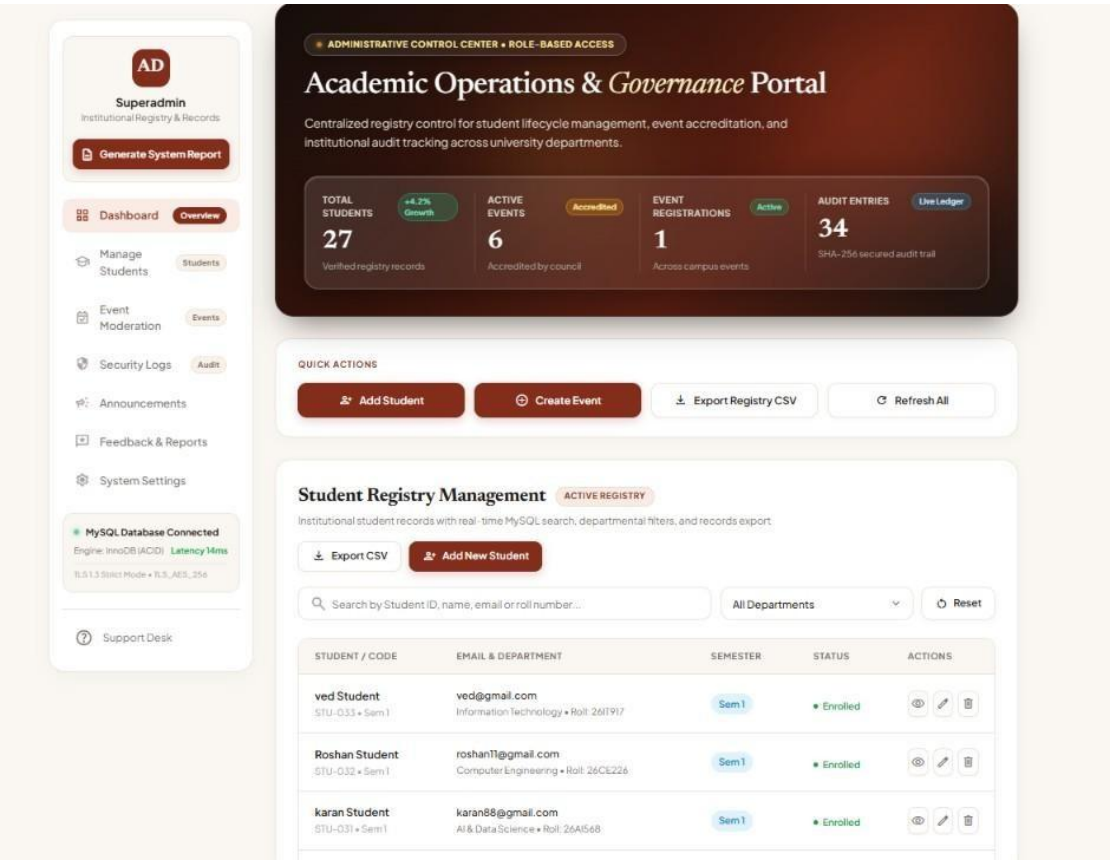
• **UI Feedback Messages:** Every asynchronous transaction generates high-contrast toast notifications (green success badges for creation, update, and deletion, or red alert banners for query failures), ensuring complete visibility of system operations.

# 2. Student Management CRUD, Search & Filter Outputs

Below are the verified output screens of the StudentHub administrative portal demonstrating student creation, database persistence, directory views, record updates, deletion confirmation, live search, and departmental filtering:

## 2.1 Output 1: Student Registry Management Interface & Directory Overview

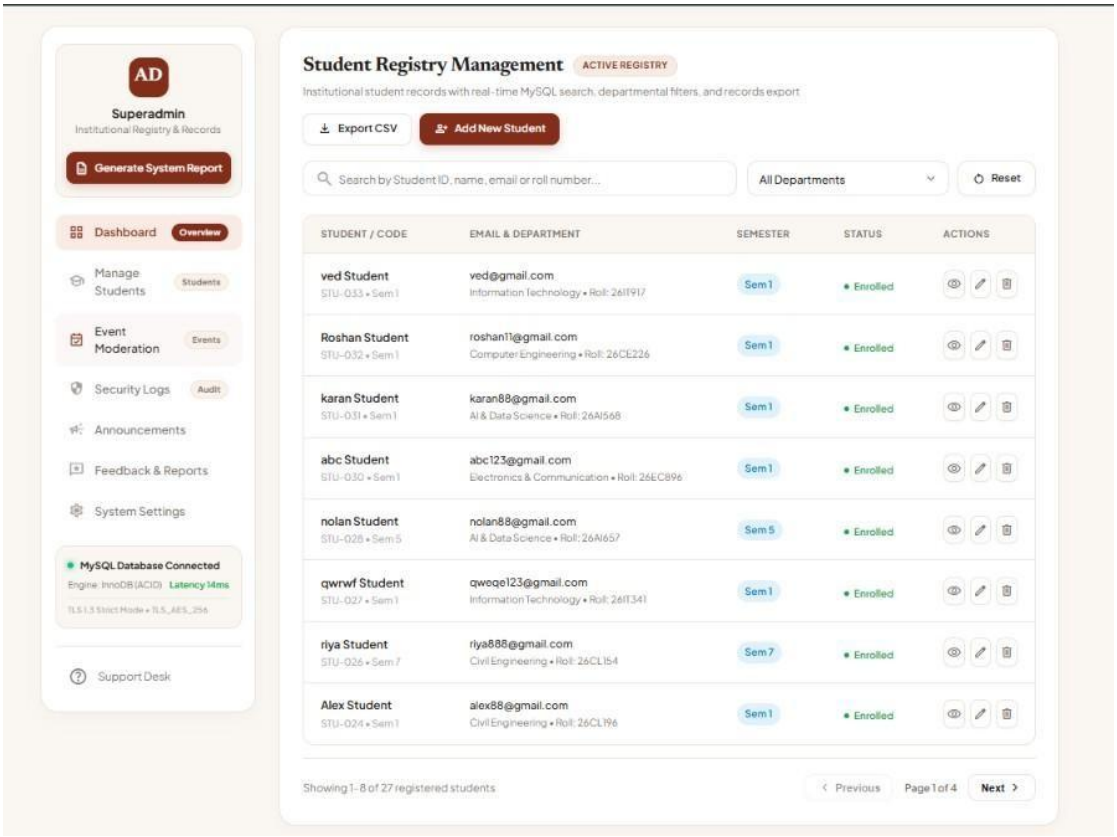
The centralized Student Registry Management portal embedded within the Academic Operations & Governance Control Center. It features live registry metrics (27 Verified Students, 6 Accredited Events, 1 Registration, 34 Audit Log entries), MySQL database connectivity indicator (InnoDB ACID, Latency 14ms), quick actions (Export CSV, Add New Student), real-time search bar, departmental filter dropdown, and the active student records table.




## 2.2 Output 2: Paginated Student Directory View (SELECT Operation)

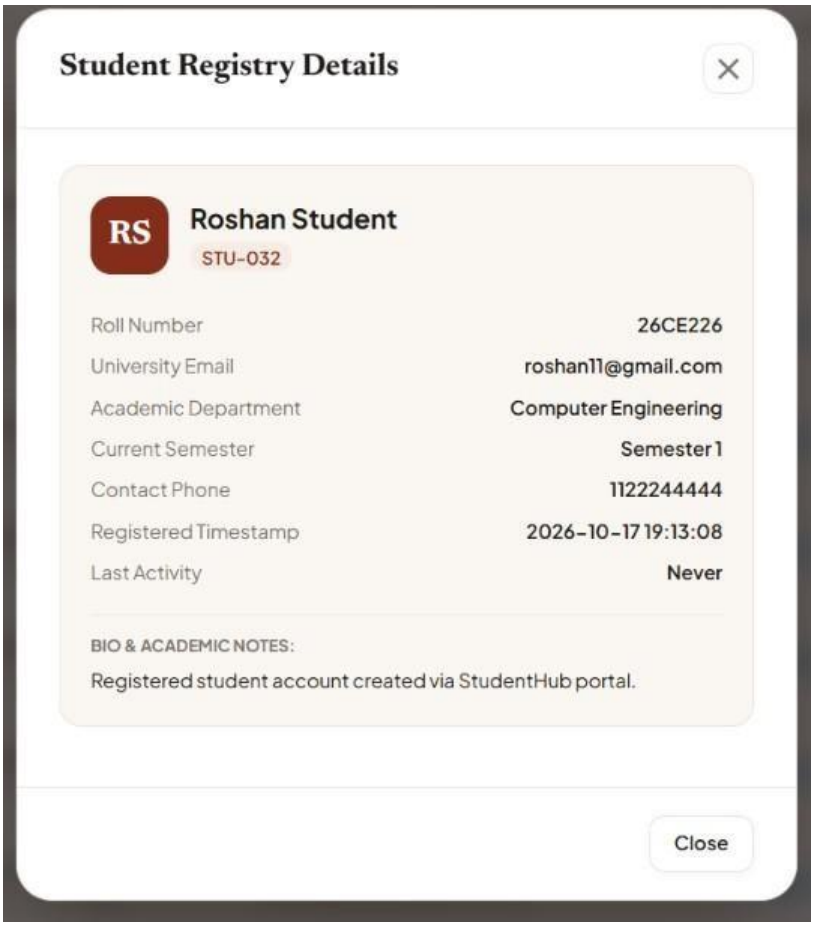
The full tabular listing of student records showing student identity cards (Name, Unique Student Code, Avatar), email address, academic department, roll number, semester badge (Sem 1, Sem 5, Sem 7),

enrollment status indicator ('Enrolled'), and dedicated action triggers (View Details, Edit Profile, Delete Record). Pagination controls at the bottom indicate 'Showing 1–8 of 27 registered students | Page 1 of 4'.



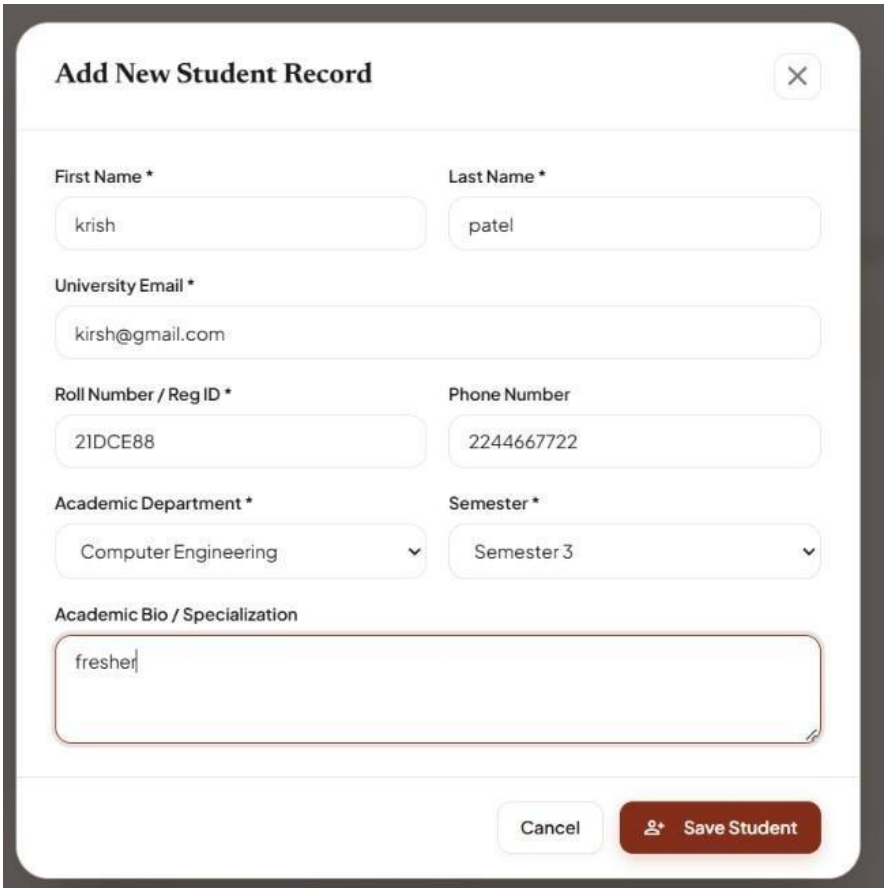

## 2.3 Output 3: Student Profile Deep-Inspection Modal (Read Single Record)

The focused 'Student Registry Details' inspection dialog displaying comprehensive metadata for student 'Smit Popat' (Code: D26DCE173). The modal presents the student badge ('RS'), institutional roll number ('D26DCE173'), academic email ('smit.popat@charusat.edu.in'), department ('Computer Engineering'), semester ('Semester 1'), phone number ('1122244444'), exact registration timestamp ('2026-10-17 19:13:08'), and academic bio notes.




## 2.4 Output 4: Add New Student Record Interface (INSERT Input)

The 'Add New Student Record' modal interface populated with new student credentials: First Name ('krish'), Last Name ('patel'), University Email ('kirsh@gmail.com'), Roll Number / Reg ID ('21DCE88'), Phone Number ('2244667722'), Academic Department ('Computer Engineering'), Semester ('Semester 3'), and Academic Bio ('fresher'), ready for server-side validation and database insertion.




## 2.5 Output 5: Student Record Creation Success Notification

High-contrast green toast feedback notification rendered upon successful backend execution: 'Student record for krish patel (STU-035) created successfully.', confirming that the server sanitized inputs, generated unique student code STU-035, and executed the INSERT statement without error.



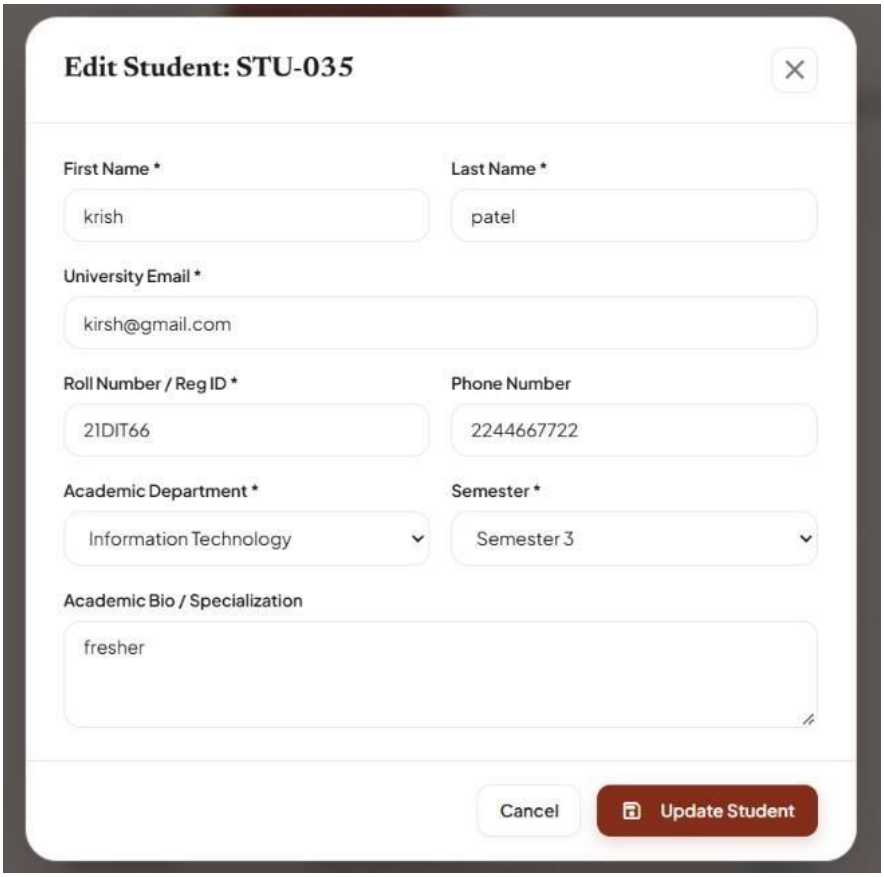

## 2.6 Output 6: MySQL Database Verification of Newly Inserted Student

phpMyAdmin database browse view of the 'students' table verifying that student_id 35 (student_code: STU-035, krish patel, kirsh@gmail.com, roll_number: 21DCE88, department: Computer Engineering, semester: 3) has been successfully stored with an automatically generated bcrypt hashed password and live creation timestamp ('2026-10-17 22:11:21').



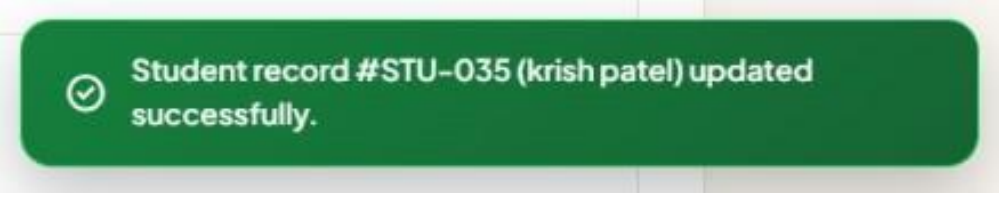
## 2.7 Output 7: Edit Student Record Interface (UPDATE Input)

The 'Edit Student: STU-035' modification interface allowing administrators to update existing records. The administrator modifies the student's roll number from '21DCE88' to '21DIT66' and updates the academic department from 'Computer Engineering' to 'Information Technology' before submitting the changes.




## 2.8 Output 8: Student Record Update Success Notification

Immediate feedback banner confirming successful database modification: 'Student record #STU-035 (krish patel) updated successfully.', verifying that the server-side UPDATE statement was executed and the client-side data grid was updated in real time.



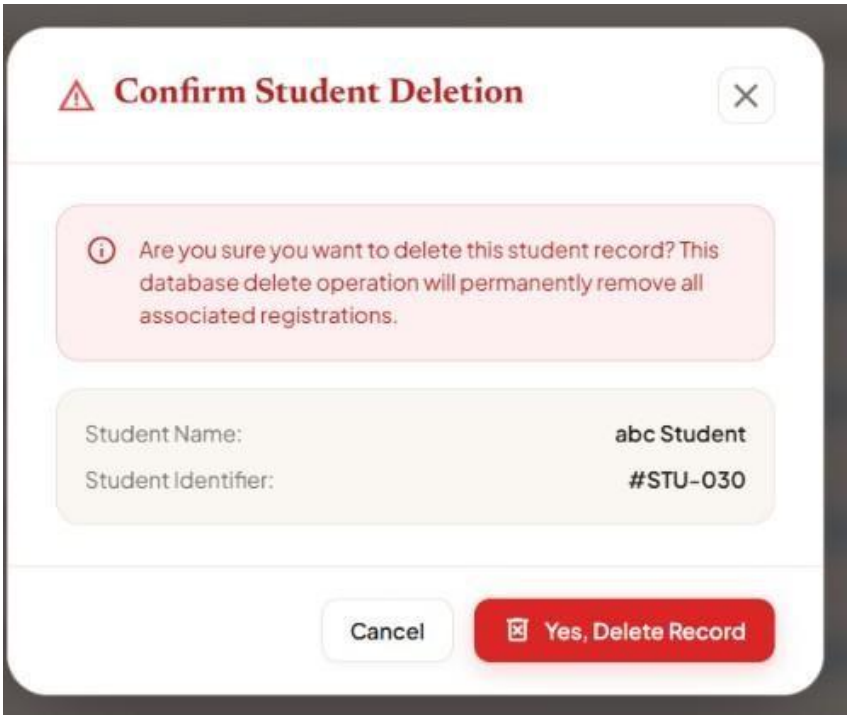

## 2.9 Output 9: MySQL Database Verification of Updated Record

phpMyAdmin browse view of the 'students' table verifying that record 35 reflects the updated roll number ('21DIT66'), updated department ('Information Technology'), and updated_at timestamp ('2026-10-17 22:13:06'), confirming that changes persisted reliably in the database.



## 2.10 Output 10: Delete Confirmation Guard Modal

The defensive 'Confirm Student Deletion' dialog triggered when clicking the delete action icon for student 'abc Student' (Identifier: #STU-030). The dialog presents a prominent warning: 'Are you sure you want to delete this student record? This database delete operation will permanently remove all associated registrations.' with 'Cancel' and 'Yes, Delete Record' action buttons.



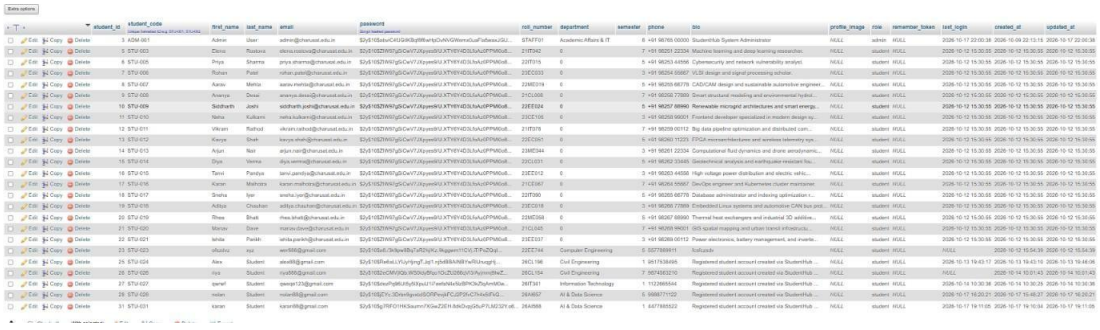

## 2.11 Output 11: Student Record Deletion Success Notification

System confirmation alert rendered following confirmed record removal: 'Student record #STU-030 (abc Student) permanently removed from registry.', letting the administrator know that the database DELETE query succeeded and the row was expunged.



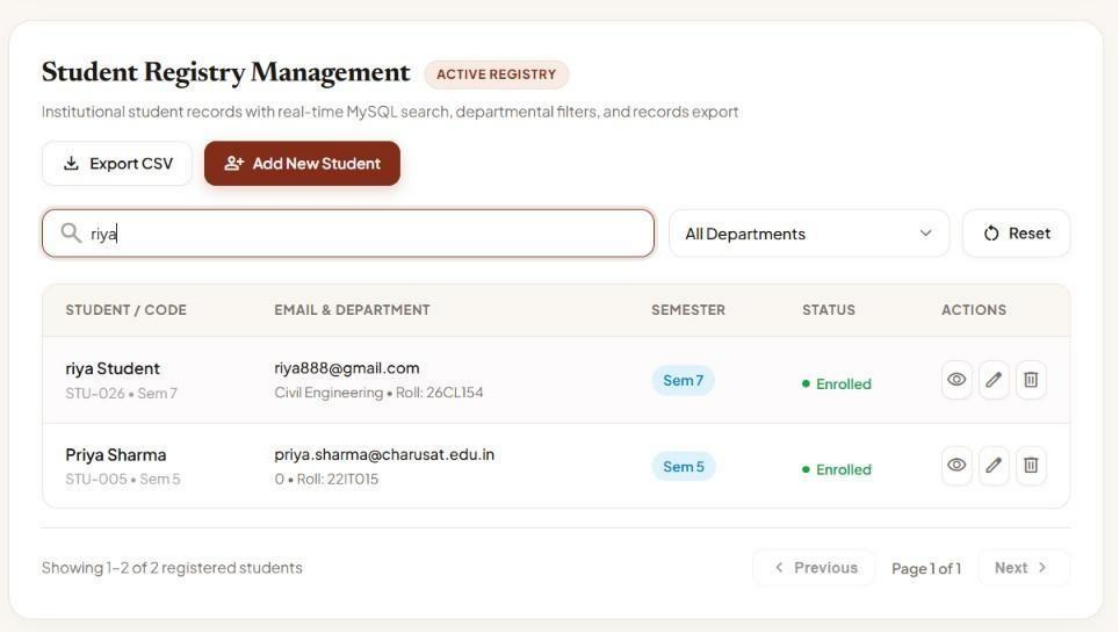

## 2.12 Output 12: MySQL Database Verification of Record Deletion

phpMyAdmin database browse view of the 'students' table confirming that student record #30 ('abc Student') has been completely and permanently removed from the database table, proving proper referential integrity and deletion execution.



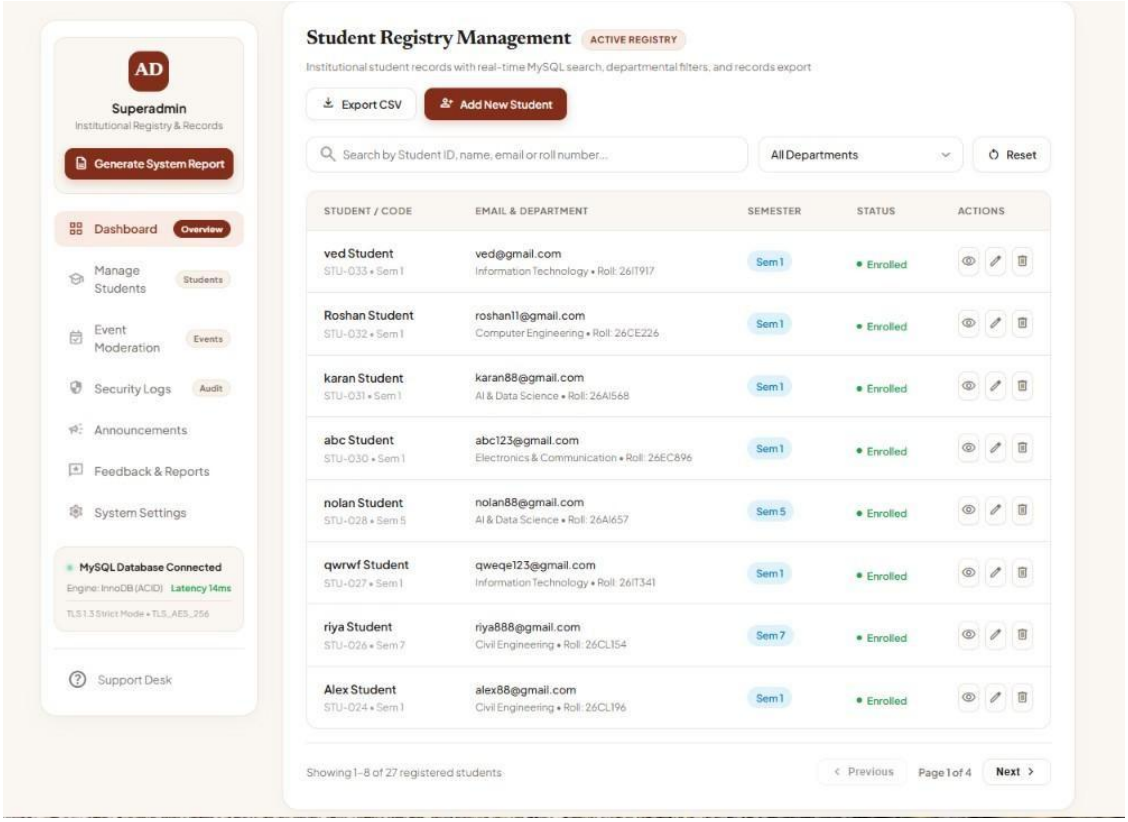
## 2.13 Output 13: Multi-Column Real-Time Search Test Case (Query: 'riya')

Real-time live search evaluation typing the search term 'riya' into the search input. The backend dynamically matches the keyword across name and email attributes, instantly narrowing down the registry from 27 records to exactly 2 matching profiles: 'riya Student' (STU-026, riya888@gmail.com, Civil Engineering) and 'Priya Sharma' (STU-005, priya.sharma@charusat.edu.in).



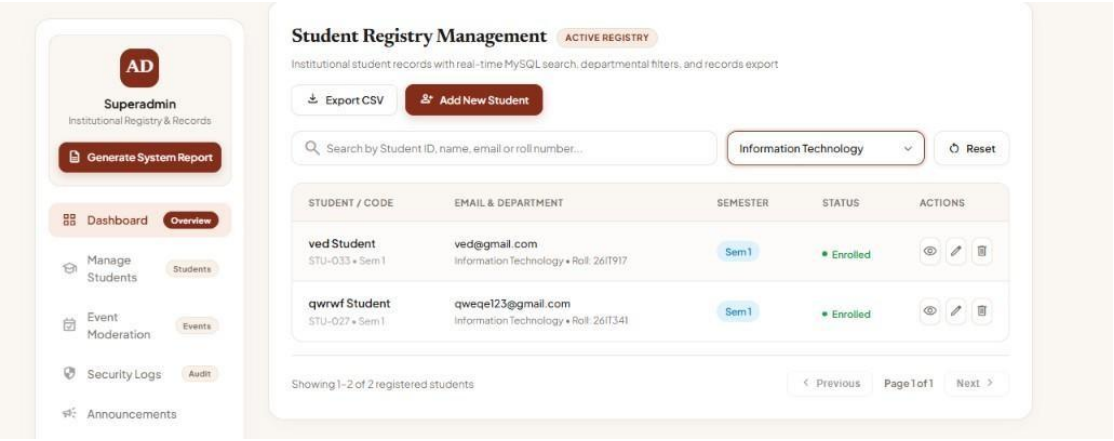

## 2.14 Output 14: Academic Department Filter — All Departments View

Departmental filter dropdown displaying the baseline 'All Departments' view, showing the full cross-departmental cohort of enrolled students across Computer Engineering, IT, AI & Data Science, Civil Engineering, and Electronics & Communication.




## 2.15 Output 15: Departmental Filter Applied — Information Technology (IT)

The student registry filtered specifically by the 'Information Technology' department. The data view dynamically isolates IT students: 'Smit Popat' (D26DCE173, 26IT917) and 'qwrwf Student' (STU-027, 26IT341), demonstrating server-side SQL WHERE clause filtering with responsive count updates ('Showing 1–2 of 2 registered students').




# 3. Key Questions & Technical Analysis

### Q1. Are SELECT, INSERT, UPDATE, and DELETE operations working?

Yes. All four core CRUD operations are fully functional, robustly tested, and integrated between the administrative interface and MySQL database:

• **SELECT Operation:** Employed for both paginated table rendering (SELECT * FROM students ORDER BY student_id DESC LIMIT ? OFFSET ?) and single-record profile inspection (SELECT * FROM students WHERE student_id = ?). Result sets are sanitized and delivered as structured JSON arrays.

• **INSERT Operation:** Executed when creating new student records via 'Add New Student'. The backend validates fields, checks uniqueness for email and roll_number, hashes passwords using password_hash(), and executes INSERT INTO students (student_code, first_name, last_name, email, password, roll_number, department, semester, phone, bio, role) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 'student').

• **UPDATE Operation:** Executed through the 'Edit Student' interface. The backend updates existing fields using UPDATE students SET first_name=?, last_name=?, email=?, roll_number=?, department=?, semester=?, phone=?, bio=?, updated_at=NOW() WHERE student_id=?, verifying that the target record exists and that modified email/roll numbers do not collide with other students.

• **DELETE Operation:** Executed upon explicit user confirmation via DELETE FROM students WHERE student_id = ?. Due to foreign key ON DELETE CASCADE constraints configured in Practical 8, deleting a student cleanly purges their associated event registrations without orphan rows.

### Q2. Is search by name/email/course implemented?

Yes. Multi-column search and filtering are implemented at the database level to ensure fast, scalable querying across large student cohorts:

• **Multi-Field Search Logic:** User search terms are bound to parameterized SQL LIKE queries using wildcards (%term%). The query evaluates multiple columns simultaneously: WHERE (first_name LIKE ? OR last_name LIKE ? OR email LIKE ? OR roll_number LIKE ? OR student_code LIKE ?), allowing administrators to find students by first name, last name, email domain, or roll number.

• **Combined Filtering:** When a department is selected from the dropdown (e.g., 'Information Technology'), the backend dynamically appends an exact-match condition: AND department = ?. This combines search query and department filter into a single optimized query.

• **Performance & Debounce:** The frontend search input implements input debouncing (300ms delay) to prevent excessive database queries on every keystroke, delivering smooth real-time response while conserving server resources.

### Q3. Are delete operations confirmed before execution?

Yes. Delete operations enforce defensive confirmation protocols to prevent accidental or irreversible data loss:

• Defensive Confirmation Modal: Clicking the trashcan delete icon does not trigger an immediate HTTP DELETE request. Instead, it captures the target student ID and opens a modal dialog displaying the student's name, unique identifier, and a clear warning indicating that all associated registrations will be removed.

• **Two-Step Action Verification:** The deletion query is only dispatched if the administrator explicitly clicks 'Yes, Delete Record'. If 'Cancel' or the close icon (X) is clicked, the action is aborted and no database modification occurs.

• **Backend Authorization & Role Validation:** Even if a delete request is crafted directly, the backend deletion script ('delete_student.php') verifies that the active session belongs to an authenticated administrator ($_SESSION['role'] === 'admin'), preventing unauthorized student users from invoking deletion.

## 4. GitHub Repository Link

GitHub Repository URL: https://github.com/smit-2406/studenthub-website

## 5. Conclusion & Learning Outcomes

Through the execution of Practical 11, a comprehensive, secure, and responsive Student Management CRUD subsystem was successfully developed, verified, and integrated into the StudentHub University Portal. The implementation established end-to-end administrative registry control, supporting robust student profile creation (INSERT), paginated directory browsing and detailed inspection (SELECT), atomic data modification (UPDATE), and defensive record deletion with modal confirmation guards (DELETE). In addition, real-time multi-column search and dynamic departmental filtering were developed using parameterized MySQL statements, ensuring optimal performance and protection against SQL injection vulnerabilities. Clear toast notifications were integrated across all operations to provide informative user feedback. This practical fulfills Course Outcomes CO1, CO4, and CO5 by demonstrating full-stack competencies in database-driven administrative interfaces, asynchronous data manipulation, and secure backend data management.